1. Introduction

This report presents a comparative study of three sorting approaches implemented in C++ to order a dataset of 100,000 words: QuickSort using a `vector`, HeapSort using a Binary Heap, and an AVL Tree-based sorting strategy. The objective is to evaluate these approaches from both theoretical and practical perspectives, considering implementation strategy, execution time, memory usage, algorithmic complexity, and overall suitability for the problem.

Sorting is a fundamental task in computer science, and its efficiency depends not only on asymptotic complexity but also on implementation details, memory behavior, and data structure overhead. For that reason, this report combines theoretical analysis with experimental measurements in order to identify the most appropriate method for the given dataset.

2. Implementation Approach

The dataset was loaded from a text file containing 100,000 words. Three different approaches were implemented from scratch in C++:

2.1 QuickSort

QuickSort is a divide-and-conquer sorting algorithm based on partitioning. At each recursive step, a pivot element is selected and the dataset is rearranged so that elements smaller than the pivot are placed on one side and larger elements on the other. The same process is then applied recursively to the resulting subarrays.

In this implementation, QuickSort was applied directly to a `vector<string>`. The algorithm was implemented recursively using a partition function, which reorganizes the vector around the chosen pivot.

From a theoretical perspective, QuickSort has average-case complexity $O(n \log n)$, although its worst-case complexity is $O(n^2)$. In practice, however, it is often one of the fastest sorting methods because of its low overhead and efficient use of contiguous memory.

2.2 HeapSort

HeapSort is based on the heap data structure. The algorithm first transforms the input into a binary max-heap, where the largest element is always stored at the root. Then, the root is repeatedly swapped with the last element, reducing the heap size step by step until the full sequence becomes sorted.

In this project, HeapSort was implemented using a binary max-heap over a `vector<string>`. The process involved first building the heap and then repeatedly applying the heapify operation after each extraction.

HeapSort has complexity $O(n \log n)$ in the best, average, and worst cases. Its main theoretical advantage is that it guarantees this bound regardless of the input. However, in practice it may be slower than QuickSort due to a larger number of swaps and less favorable memory access patterns.

2.3 AVL Tree-Based Sorting

The AVL-based approach does not sort the data directly through repeated exchanges. Instead, it inserts the elements into a self-balancing binary search tree. Because the tree remains balanced after every insertion, search and insertion operations remain efficient. Once all elements are inserted, an inorder traversal produces the words in sorted order.

In this implementation, each word from the dataset was inserted into an AVL Tree. After all insertions were completed, an inorder traversal was performed to generate the final sorted sequence.

The insertion phase requires $O(n \log n)$, and the inorder traversal requires $O(n)$, resulting in an overall complexity of $O(n \log n)$. Although this approach is efficient, it requires more memory because of the node-based structure, the balancing metadata, and the use of pointers. To evaluate performance, execution time was measured using `std::chrono`. Memory usage was estimated approximately from the data structures required by each algorithm. Correctness was verified by checking that the resulting output of each method was properly sorted.

3. Performance Measurements

3.1 Experimental Methodology

The experimental evaluation was performed on the same dataset of 100,000 words for all three algorithms in order to ensure a fair comparison. Each implementation received exactly the same input data. Execution time was measured using `std::chrono`, and approximate memory usage was estimated from the data structures employed by each algorithm. In addition, correctness was verified by checking that the resulting output was sorted in non-decreasing lexicographic order.

Although the measurements reported here correspond to a single dataset configuration, the comparison remains meaningful because the same conditions were maintained for all implementations.

3.2 Quantitative Metrics

To complement the raw measurements, additional derived metrics were computed. QuickSort was taken as the baseline because it achieved the lowest execution time.

Algorithm	Normalized Time	Time Increase	Memory Overhead	Throughput
QuickSort	1.000	0.00%	0.00%	777.58
HeapSort	1.471	47.07%	0.00%	528.72
AVL Tree	0.999	-0.15%	86.80%	778.74

These derived metrics provide a more detailed quantitative comparison. Relative to QuickSort, HeapSort required 47.07% more execution time, while AVL Tree showed a very similar runtime and was only 0.15% faster in this specific execution. In terms of throughput, AVL Tree processed approximately 778.74 words/ms, QuickSort 777.58 words/ms, and

HeapSort 528.72 words/ms. However, AVL Tree exhibited an 86.80% memory overhead compared with the array-based approaches, which confirms that its small timing advantage came at a substantially higher space cost.

The following results were obtained experimentally:

Algorithm	**Time (ms)**	**Memory (MB)**
QuickSort	128.604	4.395
HeapSort	189.136	4.395
AVL Tree	128.413	8.210

Figures 1, 2, 3, and 4 provide a visual summary of the results.

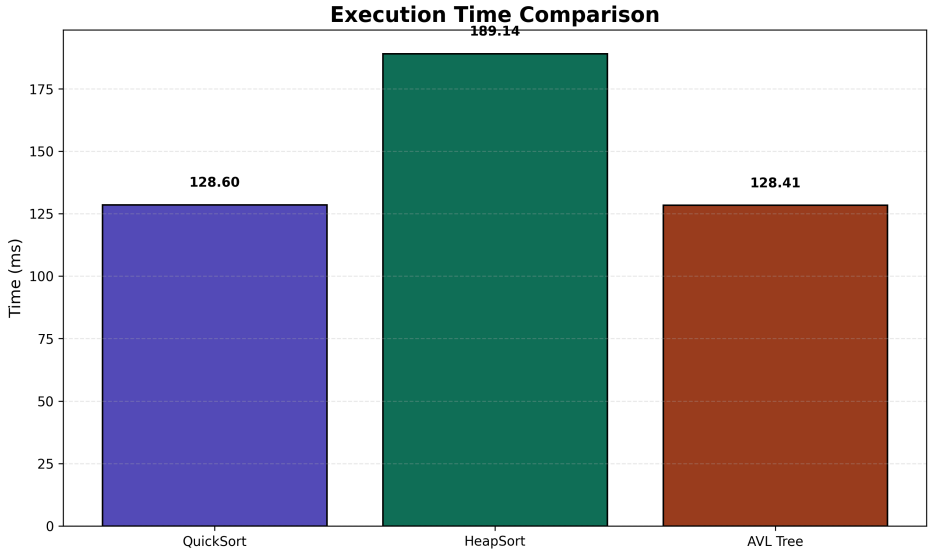

Figure 1: Execution time comparison among QuickSort, HeapSort, and AVL Tree-based sorting.

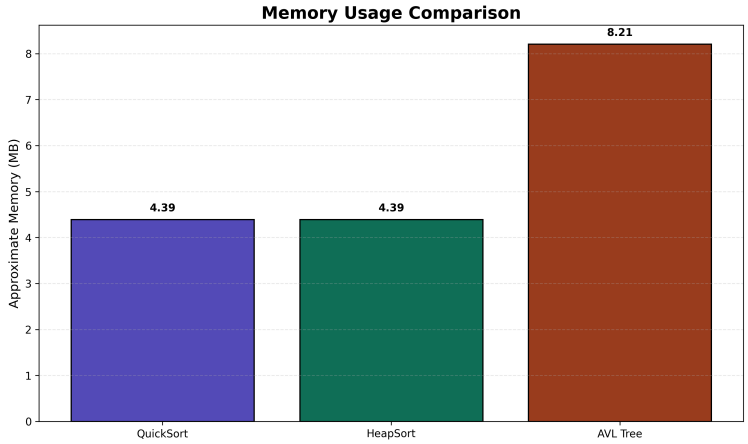

Figure 2: Approximate memory usage comparison for the three sorting approaches.

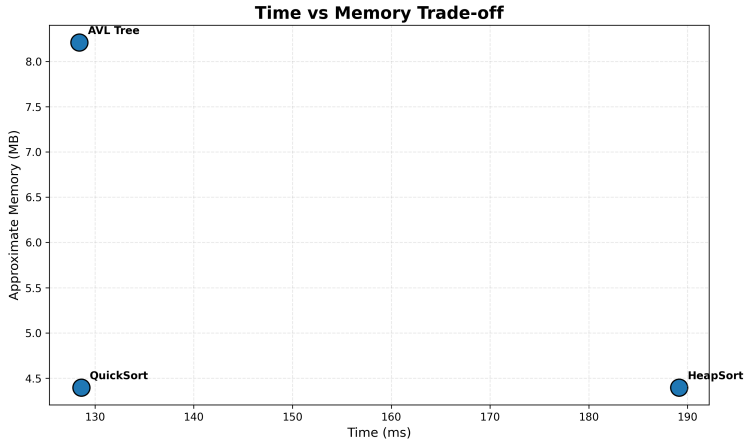

Figure 3: Time versus memory trade-off observed in the experiment.

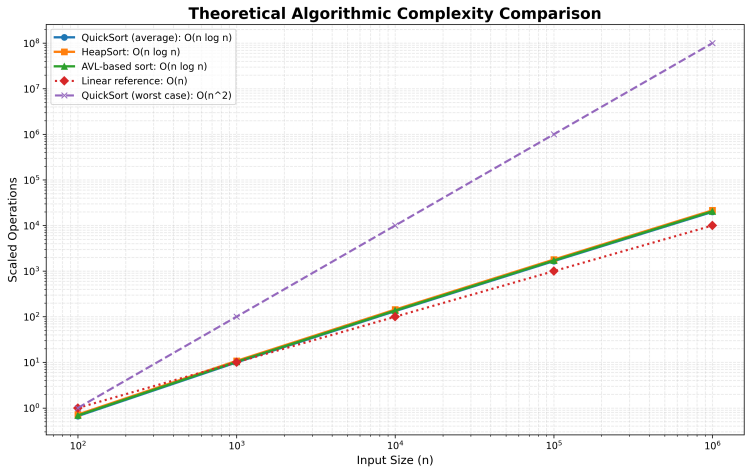

Figure 4: Theoretical algorithmic complexity comparison. QuickSort, HeapSort, and AVL-based sorting follow $O(n \log n)$ growth under standard conditions, while QuickSort may degrade to $O(n^2)$ in the worst case.

The three algorithms can be compared from both theoretical and practical perspectives.

From the standpoint of time complexity, QuickSort has average-case complexity $O(n \log n)$, but it may degrade to $O(n^2)$ in the worst case. HeapSort guarantees $O(n \log n)$ behavior in all cases, which makes it theoretically more robust. The AVL Tree-based approach also achieves overall $O(n \log n)$ complexity, since insertion into a balanced tree costs $O(\log n)$ per element and inorder traversal costs $O(n)$.

From the standpoint of space usage, QuickSort and HeapSort are more efficient for this problem because they operate directly on a contiguous vector-based representation. By contrast, the AVL Tree approach requires additional storage for nodes, pointers, height information, and balancing operations, which explains its larger memory consumption.

From the experimental perspective, QuickSort achieved the best execution time, HeapSort was the slowest, and AVL Tree produced intermediate timing results. This confirms that algorithms with similar asymptotic complexity may still differ significantly in practice

because of constant factors, recursion overhead, cache locality, and memory access patterns.

Finally, in terms of suitability for the problem, QuickSort and HeapSort are direct sorting algorithms over an array-like structure, whereas the AVL Tree approach is more naturally interpreted as an ordered data structure that can also be used to recover sorted output. For the specific task of sorting a static dataset of words, QuickSort is more appropriate because it offers the best practical performance with moderate memory requirements.

5. Conclusions

The experimental results indicate that **QuickSort** and AVL Tree were the best-performing approaches in terms of execution time, with AVL Tree being only slightly faster in this specific run. HeapSort showed the largest execution time, although it preserved its theoretical advantage of guaranteeing $O(n \log n)$ complexity in the worst case.

When memory usage is also taken into account, QuickSort emerges as the most balanced solution. While AVL Tree achieved a marginally lower execution time, it required substantially more memory because of its node-based structure and balancing metadata. QuickSort, by contrast, combined near-optimal runtime with significantly lower memory consumption.

For this reason, **QuickSort may be considered the most appropriate overall approach for this problem**, since it offers the best trade-off between speed, memory efficiency, and implementation simplicity for sorting a static dataset of words stored in a contiguous structure such as a `vector`. These results also highlight that practical algorithm selection should consider not only asymptotic complexity, but also empirical behavior and resource usage.

References

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

2. C. A. Álvarez Henao, "Estructura de Datos y Algoritmos I," GitHub repository. [Online]. Available: `https://github.com/carlosalvarezh/EstructuraDatosAlgoritmos1`

3. cppreference.com, "std::chrono::high_resolution_clock." [Online]. Available: `https://en.cppreference.com/w/cpp/chrono/high_resolution_clock`

4. cppreference.com, "std::vector." [Online]. Available: `https://en.cppreference.com/w/cpp/container/vector`

5. GeeksforGeeks, "AVL Tree Data Structure." [Online]. Available: `https://www.geeksforgeeks.org/dsa/introduction-to-avl-tree/`